

The Maximum Clique Problem in a Disk Graph Made Easy

A seminar summary of Keil and Mondal, SoCG 2025

Guy Or

The Open University of Israel

September 2026

1 Introduction

The problem. A Disk Graph is a graph where each vertex corresponds to a disk in the plane, and there is an edge between two vertices if and only if the corresponding disks intersect. The Maximum Clique problem in a disk graph asks for the largest subset of vertices such that every pair of vertices in the subset is connected by an edge, meaning that all corresponding disks intersect with each other. The central question addressed by Keil and Mondal’s paper [KM25] is whether this maximum clique problem can be computed in polynomial time for disk graphs with multiple distinct radii. The authors consider the case of k distinct radii and provide an algorithm, with runtime $O(n^{2k} \text{poly}(n))$, that is polynomial for every fixed k — placing the problem, parameterized by k , in the class XP. For the general case of arbitrary radii, the complexity of the problem remains unknown. The paper’s contribution is significant because it settles a question posed by Cabello in 2015 [Cab15]: whether the case of two distinct radii is polynomial-time solvable.

Motivation and history. In the context of geometric intersection graphs, the maximum clique problem has been a long-standing challenge, particularly for disk graphs with varying radii. The problem is known to be polynomial-time solvable for several related classes such as rectangles [IA83] and trapezoids [FMW97], yet NP-hard for others such as rays [CCL13], ellipses and triangles [AW05], grounded strings [KMMN22], and balls [BBB⁺21]. In 1990, Clark, Colbourn and Johnson [CCJ90] showed that the case of unit disk graphs (where all disks have an equal radius) can be solved in polynomial time, yet the complexity for disks of arbitrary radii has remained unresolved for over three decades. In 2015, Cabello [Cab15] explicitly posed even the case of two distinct radii as an open question. A factor-2 approximation [AW05] via stabbing all disks with four points [Dan86, CKM23] is known to be computable in linear time, and improving it below 1.99 remains open even for two radii. Disk graphs have practical applications in wireless networks [HS95], where nodes can be represented as disks with varying transmission ranges, and finding the maximum clique can help optimize network connectivity and resource allocation.

Why the obvious approaches fail. In the general case, finding a maximum clique in a graph is NP-hard, yet the geometric principle used by Clark, Colbourn and Johnson [CCJ90] for unit disks — which relies on the concept of “lenses” — allows for a polynomial-time solution: for each guessed farthest pair of disks, the candidate disks split into two cliques that together are guaranteed to contain the maximum clique. The maximum clique can then be recovered by running a maximum matching algorithm — itself polynomial — on the complement of the candidate graph. The naive approach of applying the unit disk algorithm to each radius type separately and then combining the results fails because the maximum clique can contain disks of different radii, and thus is not guaranteed to be contained within any single radius type.

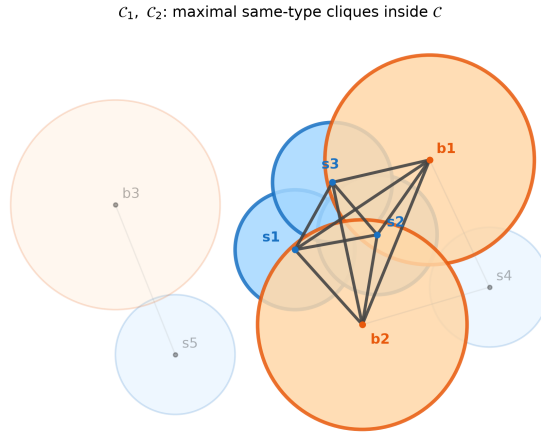


Figure 1: The maximum clique contains disks of both types, it is not contained in any single type class. (Figure by the author; see Appendix 5.)

Attempting to generalize the lens-based approach to multiple radii also fails, as the existence of distinct “clique regions” within the lens is no longer guaranteed. Keil and Mondal demonstrate this in Section 6.1 of their paper by examining several natural lens-based attempts and showing that they fail to yield the two-clique split that the reduction requires.

Results and roadmap. The main result of the paper is given in **Theorem 3.2**: A maximum clique in a disk graph with k radius types and n vertices is computable in $O(n^{2k}(f(n) + n^2))$ time, where $f(n)$ is the maximum matching time in a bipartite graph with n vertices. Specifically, for $k = 2$ (and any fixed k) the runtime is polynomial, settling Cabello’s question. The remainder of this summary is organized as follows. **Section 2** introduces the notation and the graph-theoretic tools used later. **Section 3** presents the guessing framework and why the seemingly trivial step of choosing the anchors is, in fact, the heart of the problem. **Section 4** describes the algorithm: the geometric lemma, the construction of the slab, and the reduction to bipartite matching, illustrated on a running example. **Section 5** discusses the complexity-theoretic status of the result, subsequent work, and open problems. Keil and Mondal present in the paper two more important results — they generalized Theorem 3.2 to ball graphs with k distinct radii whose centers lie on r parallel planes (later

relaxed to r planes perpendicular to a common plane), and achieved a speed-up of at least a factor $n^{4/3}$ for precomputing a maximum clique in *unit* disk graphs, for every axis-aligned rectangle determined by the disk centers, so that rectangular range queries can be answered by table look-up. This summary focuses only on the first main result since it is the paper's main contribution, while the other two results are further applications of the same slab-based approach they introduce and its properties.

2 Preliminaries

Notation. We write $D_{a,r}$ for a disk with center a and radius r . For a pair of points a and b , ℓ_{ab} is the line through them, ab is the line segment with endpoints a and b , and $|ab|$ is the Euclidean distance between them. For a point p , ℓ_p^h and ℓ_p^v are the horizontal and vertical lines through p , respectively.

Concepts.

Disk graph. A disk graph G is an intersection graph of disks in $\mathbb{R}^2$: each vertex of G corresponds to a disk, and two vertices are adjacent in G if and only if their disks intersect.

Lens. Given a pair of disks $D_{a,r}$ and $D_{b,r'}$, their common intersection region is called a *lens*, denoted $L(D_{a,r}, D_{b,r'})$.

Upper and lower slabs. The *upper slab* U_{ab} of a segment ab is the region above ab bounded by the lines ℓ_a^v and ℓ_b^v ; the *lower slab* $\bar{U}_{ab}$ is the region below ab bounded by the same two lines. Equivalently, a slab is an intersection of three half-planes.

Radius types. $\mathcal{D}_k$ is a disk graph in which the number of different radii is at most k . We denote the radii by $r_1 < r_2 < \dots < r_k$, and a *type- i disk* is a disk of radius r_i .

Cliques. Given a disk graph $\mathcal{D}_k$: C_i is a clique in $\mathcal{D}_k$ containing only type- i disks, $\mathcal{C}$ is a maximum clique in $\mathcal{D}_k$, and $\mathcal{C}_i$ denotes the type- i disks of $\mathcal{C}$.

Basic facts and graph-theoretic tools.

Intersection condition. Two disks $D_{a,r}$ and $D_{b,r'}$ intersect if and only if the distance between their centers is at most the sum of their radii: $|ab| \leq r + r'$.

(T1) Complement. For a graph G , a set of vertices is a clique in G if and only if it is an independent set in the complement $\bar{G}$; hence $\omega(G) = \alpha(\bar{G})$.

(T2) Co-bipartite graphs. A graph is *co-bipartite* if its vertex set is the union of two cliques. Its complement is bipartite, since neither clique contributes edges to the complement.

(T3) Gallai's identity [Gal59]. For any graph H on m vertices, $\alpha(H) = m - \tau(H)$, where $\tau(H)$ is the size of a minimum vertex cover.

(T4) **König’s theorem** [Kön31, KT06]. For a *bipartite* graph H , $\tau(H) = \nu(H)$, where $\nu(H)$ is the size of a maximum matching. Together with (T3), $\alpha(H) = m - \nu(H)$, which is computable in polynomial time. We write $f(n)$ for the time to compute a maximum matching in an n -vertex bipartite graph, e.g. via Hopcroft–Karp [HK73].

3 The Guessing Framework

Defining the certificate. The main idea of the algorithm is to guess the “anchors” of the maximum clique $\mathcal{C}$, which are the *leftmost* and *rightmost* disks of **each type** in $\mathcal{C}$. This guessed set of disks, denoted by Ψ , is the *certificate* of the solution and given Ψ — and the descriptions of the disks — the clique can be reconstructed in polynomial time from them. The reason that the correct guess for Ψ solves the *hard* part of the problem is as follows — consider a disk D of type i that is in the maximum clique $\mathcal{C}$, but is not an anchor, and let D_{a_i, r_i} and D_{b_i, r_i} be the leftmost and rightmost disks of type i in $\mathcal{C}$, respectively. Then the center of D must lie *between* the centers of D_{a_i, r_i} and D_{b_i, r_i} in the x -direction, specifically either in the slab $U_{a_i b_i}$ or in the slab $\bar{U}_{a_i b_i}$ and since all disks in $\mathcal{C}$ intersect with each other, D must intersect with both D_{a_i, r_i} and D_{b_i, r_i} .

The cost of guessing. Ψ contains **at most** $2k$ disks, because there are k types and **each type** contributes at most two anchor disks. Thus, the number of possible *guesses* for Ψ is

given by $\left[\underbrace{\binom{n}{0}}_{\text{type absent}} + \underbrace{\binom{n}{1}}_{a_i=b_i} + \underbrace{\binom{n}{2}}_{a_i \neq b_i} \right]^k = O(n^{2k})$. The inner sum counts the number of ways to

choose the anchors for a single type: either no disks of that type are present in $\mathcal{C}$, or one disk is present (in which case it is both the leftmost and rightmost), or two disks are present (the leftmost and rightmost). This factor is raised to the power of k because the choices for each type are independent of the others. This guessing “loop” is what contributes to the algorithm its dominating component of $O(n^{2k})$. Finally, this method represents an *improvement* over a naive exhaustive search for the clique itself: enumerating the *certificate* costs $n^{O(k)}$, whereas enumerating candidate cliques directly is exponential in n . It is the **size** of the certificate — at most $2k$ disks, regardless of how large $\mathcal{C}$ is — that makes the cost affordable.

Soundness and Completeness. The algorithm returns the maximum clique by means of keeping the largest clique resulting from all the guesses. A core question that is worth asking to validate the correctness of this method is whether this combinatorial scan is *sound* — i.e. does it only consider valid cliques — and whether it is *complete* — i.e. is it guaranteed to encounter and consider a maximum clique in the input graph. To prove *completeness* it is simple to observe that by the definition of the optimal Ψ it is always the case that there exist a leftmost and a rightmost disk of any given disk-type that participates in the maximum clique. Because the authors allowed for the leftmost and rightmost to be the same disk, it is also true for edge cases where there is only one disk of a certain disk-type in the clique or where all of their centers have the same x coordinate. Since the enumeration covers every such choice, some iteration guesses exactly this Ψ . As shown above, for this guess, every disk

of $\mathcal{C}$ is either in Ψ or, is trapped in its type's slab and intersects all of Ψ , so the construction of Section 4 recovers a clique of size $|\mathcal{C}|$.

To prove *soundness* we must be sure that the algorithm never considers a set of vertices that is not a clique. This is achieved by verifying for each guessed Ψ that it is a clique itself, which costs $O(k^2)$ per guess. The rest of the algorithm — the “inner loop” that constructs the clique from the given guess for Ψ — is proved to be sound by Lemma 3.1. Lemma 3.1's proof only assumes that a disk lies in its type's slab and intersects every disk of Ψ . It never assumes that the guess is correct. Hence every valid guess yields a genuine clique. Incorrect guesses simply produce smaller ones and lose the final maximum.

The core innovation and the need for search. The core innovation presented by Keil and Mondal in this paper is the definition of Ψ and the sound use of the slabs as a means for constructing the clique. The paper did not discover an efficient way to *find* Ψ , and therefore settles for exhaustively enumerating all options, with the resulting time complexity nonetheless being the first polynomial-time algorithm for any fixed number of radii. An efficient method for finding Ψ would immediately improve the running time, and might bring the general case within reach — but with an unbounded number of radii the certificate itself may be of size $\Theta(n)$, so this remains open as of writing.

4 The Algorithm

To find a maximum clique in a disk graph $\mathcal{D}_k$, given the description of the disks, the algorithm presented in the paper works as follows:

Guess. For each choice of at most two disks per radius type — possibly none, and possibly the same disk twice — let Ψ be the chosen disks, and do:

Validity-check. If Ψ is not a clique continue to the next guess.

Create the Slabs. Otherwise construct the slabs defined by each pair of same-type disks in Ψ .

Filter. For each disk type i that is present in Ψ : Let X_i be the set of all *type- i* disks in $\mathcal{D}_k \setminus \Psi$ whose centers are in the **upper** slab defined by the *type- i* anchors in Ψ AND are intersecting all disks in Ψ . Let Y_i be the set of all *type- i* disks in $\mathcal{D}_k \setminus \Psi$ whose centers are in the **lower** slab defined by the *type- i* anchors in Ψ AND are intersecting all disks in Ψ . Let $X = X_1 \cup \dots \cup X_k, Y = Y_1 \cup \dots \cup Y_k$.

Create the max clique in $X \cup Y$. Let H be the complement of the graph induced by $X \cup Y$, compute the max-independent-set in H using a max-matching algorithm (e.g. Hopcroft–Karp [HK73]) and let the result be called $MIS_{X \cup Y}$.

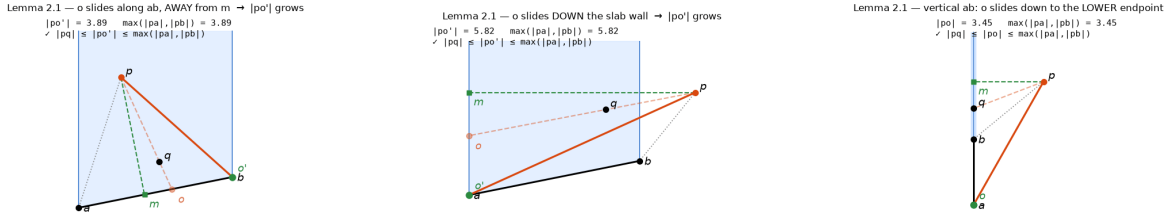
Assemble. Let $\hat{\mathcal{C}} = \Psi \cup MIS_{X \cup Y}$ be the candidate solution.

Aggregate. Return the maximum $\hat{\mathcal{C}}$ found, by keeping a running maximum over all candidate solutions.

4.1 The geometric basis: Lemma 2.1

Lemma 2.1 Let ab be a line segment and let U_{ab} be the upper slab of ab . Let q be a point in U_{ab} and let p be a point (not necessarily in U_{ab}) with a y -coordinate equal to or larger than the y -coordinate of q . Then $|pq| \leq \max\{|pa|, |pb|\}$.

Lemma 2.1 provides the basic geometric principle which allows the authors of the paper to leverage the slabs to construct the clique. The proof discusses two basic scenarios with a total of three distinct cases for which the lemma is shown to be true. In the first scenario, the x -coordinates of a and b differ and therefore the slab is a two-dimensional area. This scenario is further split into two cases — one in which a ray that starts at p and goes through q intersects the slab at the line segment ab in a point denoted o . In the second case, the same ray intersects the slab at one of its sides. In the first case “sliding” the intersection point o away from the point m — where pm is the line segment perpendicular to ab — causes $|po|$ to monotonically get longer, up to $\max\{|pa|, |pb|\}$ and since $|pq| \leq |po|$ the lemma inequality follows. In the second case, pm is the line segment perpendicular to the side of the slab which the ray intersects. In that case, sliding o downwards and away from m has the same monotonic effect and the same bound holds. In the second scenario, the x -coordinates of a and b are equal and then the slab collapses into a vertical line, in which case the inequality of the lemma follows directly.



(a) o exits through ab and slides away from m . (b) o exits through a vertical side and slides down. (c) The degenerate case: a and b share an x -coordinate.

Figure 2: Illustration of Lemma 2.1: in every case the exit point o moves away from the point of the boundary closest to p , so $|po|$ grows monotonically until it reaches a or b . (Figures by the author; see Appendix 5.)

4.2 A worked example

The figure below illustrates an example iteration of the algorithm on a simple arrangement of disks with $k = 2$. Here Ψ is the correct guess — the anchors of the true maximum clique — and from it the slabs are constructed and the sets X, Y are created from the disks of the arrangement that are not in Ψ . Note that Y is not empty: it contains s_6 , which survives the filter but does not intersect $s_3 \in X$, so $X \cup Y$ is not a clique. As we will see in Section 4.4, the maximum clique found in $X \cup Y$ therefore excludes s_6 , and the clique reported for this iteration consists of Ψ and X only.

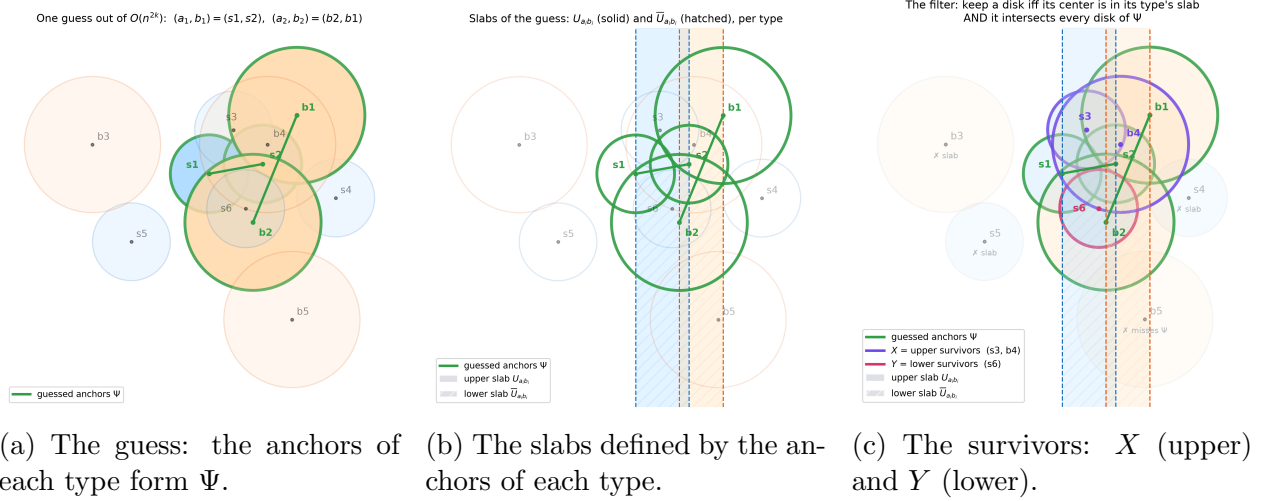


Figure 3: One iteration of the algorithm on the running example. A disk survives the filter if its center lies in its own type’s slab *and* it intersects every disk of Ψ . (Figures by the author; see Appendix 5.)

4.3 X and Y form two cliques: Lemma 3.1

Lemma 3.1 For every i, j , where $1 \leq i, j \leq k$, the disks in $X_i \cup X_j$ are mutually adjacent.

What Lemma 3.1 actually states is that both X and Y as defined in the algorithm, are cliques in the disk graph: any two disks of X lie in some X_i and X_j , so pairwise adjacency for all i, j gives that X is a clique, and Y follows by the mirrored argument on the lower slabs. The proof is as follows:

Let i, j be indices with $1 \leq i, j \leq k$ and let $D_{p,r_i} \in X_i$ and $D_{q,r_j} \in X_j$ — we show their centers satisfy $|pq| \leq r_i + r_j$, so they intersect. The claim is symmetric in the two disks, so we may name the lower one q , i.e. $y(q) \leq y(p)$. We consider the slab $U_{a_j b_j}$ constructed from the disk centers $D_{a_j, r_j}, D_{b_j, r_j} \in \Psi$. Since $D_{q, r_j} \in X_j$, its center lies in $U_{a_j b_j}$, and since $y(q) \leq y(p)$ it follows from Lemma 2.1 that $|pq| \leq \max\{|pa_j|, |pb_j|\}$. Now since D_{p, r_i} intersects both D_{a_j, r_j} and D_{b_j, r_j} (because $p \in X_i$, and every disk that survives the filter intersects every disk of Ψ) it follows from the intersection condition that $\max\{|pa_j|, |pb_j|\} \leq r_i + r_j$. And so $|pq| \leq r_i + r_j$, therefore from the intersection condition it follows that D_{p, r_i} and D_{q, r_j} intersect. ■

It is important to observe that Lemma 3.1 does not assume that Ψ is the correct guess, nor that it is a subset of $\mathcal{C}$ — only that p and q satisfy the two filter conditions: lying in their own type’s slab and intersecting every disk of Ψ . Therefore it is *sound* — X, Y are indeed cliques even for a suboptimal (but valid) guess of Ψ . The validity check on Ψ is needed elsewhere: it is what makes the assembled candidate $\Psi \cup MIS_{X \cup Y}$ a clique.

4.4 From two cliques to one matching

At this point the algorithm computes the maximum clique inside the candidate graph induced by $\Psi \cup X \cup Y$. The maximum clique in the graph induced by $X \cup Y$ is the maximum independent set (“MIS”) in the complementary graph H (T1), where $V(H) = X \cup Y$. The

algorithm finds the MIS as follows: first from Gallai’s identity (T3) it is known that $\alpha(H) = |X \cup Y| - \tau(H)$ — where $\alpha(H)$ is the size of the MIS in H and $\tau(H)$ is the size of the minimum vertex cover in H . Given that X, Y are cliques it follows that H is a bipartite graph (T2) and from König’s theorem on bipartite graphs it follows that the minimum vertex cover in H is exactly the maximum matching in H (T4). The maximum matching in H is computed via a maximum matching algorithm (such as Hopcroft–Karp [HK73], which runs in polynomial time for bipartite graphs), from which the independent set itself is recovered by an alternating-path search. This results in the maximum clique in $X \cup Y$, denoted here as $MIS_{X \cup Y}$. Since any disk in $X \cup Y$ intersects all the disks in Ψ by construction and since Ψ is a clique, it follows that $\Psi \cup MIS_{X \cup Y}$ is a clique in the graph induced by $\Psi \cup X \cup Y$. It is also the maximum clique in it: any clique K in the candidate graph satisfies $|K| \leq |\Psi| + |K \cap (X \cup Y)| \leq |\Psi| + |MIS_{X \cup Y}|$, since $K \cap (X \cup Y)$ is a clique in the graph induced by $X \cup Y$.

4.5 Running time

The outer loop of the algorithm goes through $\left[\binom{n}{0} + \binom{n}{1} + \binom{n}{2}\right]^k = O(n^{2k})$ iterations — this is the guessing part. For each iteration the Ψ pairwise intersection check costs $O(k^2)$, the filter operation costs $O(nk)$ (every disk checked for its center within its type’s slab plus that it intersects all of the at most $2k$ disks in Ψ), building the graph induced by $X \cup Y$ costs $O(n^2)$ and running the maximum matching costs $f(n) = \text{poly}(n)$. Since $k \leq n$, the $O(k^2)$ and $O(nk)$ terms are absorbed by $O(n^2)$, and so the total runtime is $O(n^{2k}(f(n) + n^2))$.

Theorem 3.2 Given a set of n disks in the Euclidean plane with k different types of radii, a maximum clique in the corresponding disk graph can be computed in $O(n^{2k}(f(n) + n^2))$ time. Here $f(n)$ is the time to compute a maximum matching in an n -vertex bipartite graph.

5 Conclusions and Discussion

In this paper, Keil and Mondal [KM25] have shown a method to compute the maximum clique in a disk graph with k radius types in $O(n^{2k}(f(n) + n^2))$ run time. The algorithm is polynomial for any fixed k — settling Cabello’s question from 2015 [Cab15] — placing the problem, parameterized by k , in the complexity class **XP**. For an unbounded number of radii the question of the true complexity remains open.

This result represents a significant breakthrough in exact methods for the maximum clique problem in disk graphs: the last time a new class of disk graphs was shown to admit an exact polynomial-time algorithm was in 1990, by Clark, Colbourn and Johnson [CCJ90] for unit disk graphs (the running time for that case has since been improved, most recently in [EKM23]).

The authors re-used the same framework as the unit disk solution — exhaustively enumerating candidate *certificates*, constructing a co-bipartite candidate graph, and extracting the clique with a maximum matching algorithm. Their breakthrough is the geometric structure that replaces the lens: the slab, whose properties restrict the search space to $O(n^{2k})$

certificates while the rest of the algorithmic framework remains exactly as in Clark, Colbourn and Johnson’s method from 1990.

Recent improvements Gao et al. [GGG⁺26] have since given randomized approximation algorithms for both settings. Both succeed with constant probability, which can be amplified by repetition. For *unit disk graphs* they showed an algorithm that computes a $(1 - \varepsilon)$ -approximate maximum clique in *expected* time $\tilde{O}(n/\varepsilon^2)$ — where $\tilde{O}$ suppresses polylogarithmic factors. For disk graphs with k distinct radii, they showed a parameterized approximation scheme that computes a $(1 - \varepsilon)$ -approximate maximum clique in expected time $\tilde{O}(f(k) \cdot (1/\varepsilon)^{O(k)} \cdot n)$ for some exponential function $f(k)$ — that is, the dependence on the number of radii is exponential while the dependence on n remains near-linear. These results trade exactness and determinism for speed, so the exact complexity question for disk graphs with unbounded radii remains open.

References

- [AW05] Christoph Ambühl and Uli Wagner. The clique problem in intersection graphs of ellipses and triangles. *Theory of Computing Systems*, 38(3):279–292, 2005.
- [BBB⁺21] Marthe Bonamy, Édouard Bonnet, Nicolas Bousquet, Pierre Charbit, Panos Giannopoulos, Eun Jung Kim, Paweł Rzazewski, Florian Sikora, and Stéphan Thomassé. EPTAS and subexponential algorithm for maximum clique on disk and unit ball graphs. *Journal of the ACM*, 68(2):9:1–9:38, 2021.
- [Cab15] Sergio Cabello. Maximum clique for disks of two sizes. Open problems from the Geometric Intersection Graphs workshop, CG Week, Eindhoven, 2015.
- [CCJ90] Brent N. Clark, Charles J. Colbourn, and David S. Johnson. Unit disk graphs. *Discrete Mathematics*, 86(1–3):165–177, 1990.
- [CCL13] Sergio Cabello, Jean Cardinal, and Stefan Langerman. The clique problem in ray intersection graphs. *Discrete & Computational Geometry*, 50(3):771–783, 2013.
- [CKM23] Paz Carmi, Matthew J. Katz, and Pat Morin. Stabbing pairwise intersecting disks by four points. *Discrete & Computational Geometry*, 70(4):1751–1784, 2023.
- [Dan86] Ludwig Danzer. Zur lösung des gallaischen problems über kreisscheiben in der euklidischen ebene. *Studia Scientiarum Mathematicarum Hungarica*, 21(1–2):111–134, 1986.
- [EKM23] Jared Espenant, J. Mark Keil, and Debajyoti Mondal. Finding a maximum clique in a disk graph. In *Proceedings of the 39th International Symposium on Computational Geometry (SoCG)*, volume 258 of *LIPIcs*, pages 30:1–30:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023.

- [FMW97] Stefan Felsner, Rudolf Müller, and Lorenz Wernisch. Trapezoid graphs and generalizations, geometry and algorithms. *Discrete Applied Mathematics*, 74(1):13–32, 1997.
- [Gal59] Tibor Gallai. Über extreme punkt- und kantenmengen. *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, 2:133–138, 1959.
- [GGG⁺26] Jie Gao, Paweł Gawrychowski, Panos Giannopoulos, Wolfgang Mulzer, Satyam Singh, Frank Staals, and Meirav Zehavi. Near-linear and parameterized approximations for maximum cliques in disk graphs. In *Proceedings of the 20th Scandinavian Symposium on Algorithm Theory (SWAT)*, LIPIcs, 2026.
- [HK73] John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973.
- [HS95] Mark L. Huson and Arunabha Sen. Broadcast scheduling algorithms for radio networks. In *Proceedings of MILCOM’95*, volume 2, pages 647–651. IEEE, 1995.
- [IA83] Hiroshi Imai and Takao Asano. Finding the connected components and a maximum clique of an intersection graph of rectangles in the plane. *Journal of Algorithms*, 4(4):310–323, 1983.
- [KM25] J. Mark Keil and Debajyoti Mondal. The maximum clique problem in a disk graph made easy. In *Proceedings of the 41st International Symposium on Computational Geometry (SoCG)*, volume 332 of *LIPIcs*, pages 63:1–63:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. Full version: arXiv:2404.03751.
- [KMMN22] J. Mark Keil, Debajyoti Mondal, Ehsan Moradi, and Yakov Nekrich. Finding a maximum clique in a grounded 1-bend string graph. *Journal of Graph Algorithms and Applications*, 26(4), 2022.
- [Kön31] Dénes König. Gráfok és mátrixok. *Matematikai és Fizikai Lapok*, 38:116–119, 1931.
- [KT06] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison Wesley, 2006.

Appendix - Declaration on the Use of AI Tools

In preparing this seminar (presentation and summary) I used Anthropic’s Claude (Fable 5 and Opus 5 models) as well as Google’s Gemini (3.1 Pro model) as follows:

- Exploring SoCG proceedings and selecting the paper.
- Discussing the paper’s proofs and testing and challenging my understanding through asking questions, answering AI-generated quizzes and iteratively checking my understanding of the different aspects of the paper and its related background material by stating key mathematical results and asking the AI to find errors or inaccuracies in them.

- Generating the Python and matplotlib code that renders the original figures in this document and in the presentation slides (the examples and their computed cliques are verified programmatically; code can be found at <https://github.com/guyo13/openu-cg-seminar>).
- Searching for related subsequent work.
- Planning the structure of the presentation slides and the summary.
- Improving the English phrasing of text I wrote and reviewing the accuracy and the coherence of the mathematical statements I wrote.
- Adding citations where needed and making sure all the bibtex entries are correct.